Fundamentals of Natural Language Processing

Part 1: NLP Landscape and Text Representation

Aymen Ben Brik

M1 GAMMA & L2 LMAD — Data Science

May 6, 2026

Course outline

Chapter 1 — NLP Landscape

1. What is NLP? Core challenges
2. Major NLP tasks and applications
3. NLP within the AI ecosystem
4. Discriminative vs. generative models
5. Historical evolution

Chapter 2 — Text Representation & Evaluation

6. Tokenization fundamentals
7. Sparse representations: BoW, TF–IDF
8. Dense representations: Word2Vec, GloVe, FastText
9. Semantic similarity
10. Evaluation metrics

Learning objectives

By the end of this course, you will be able to:

1. **Define** NLP and identify its core challenges.
2. **Recognize** the major NLP tasks and their applications.
3. **Situate** NLP within the broader AI landscape.
4. **Distinguish** discriminative from generative modelling.
5. **Trace** the evolution from rules to LLMs.
6. **Apply** tokenization and vector-space representations.
7. **Compute** evaluation metrics adapted to each task.

Chapter 1

NLP Landscape

Definition, tasks, paradigms and historical evolution

What is Natural Language Processing?

Definition

NLP is the subfield of AI that builds systems able to **process**, **understand** and **generate** human language. It sits at the intersection of *linguistics*, *computer science* and *machine learning*.

Three axes:

- **NLU** — understanding: extract meaning, entities, sentiment.
- **NLG** — generation: produce fluent, coherent text.
- **Interaction** — dialogue and conversational agents.

Why is natural language hard?

Ambiguity

- **Lexical**: "*bank*" (money / river)
- **Syntactic**: "*I saw the man with the telescope.*"
- **Semantic**: "*The chicken is ready to eat.*"
- **Pragmatic**: "*Can you pass the salt?*"

Context dependence

- Same word, different referents (*Apple*)
- Pronoun resolution
- Sarcasm, irony

Variability

- Synonymy, paraphrasing
- Spelling variation, slang
- Code-switching, dialects

The headline that everyone misreads

Real headline:

"Stolen painting found by tree."

Two grammatical readings:

1. *found by [tree]* — a tree found the painting (absurd).
2. *found [by tree]* — the painting was found beside a tree (likely).

Lesson

World knowledge (trees are inanimate) lets us discard reading 1. A computer without such knowledge cannot disambiguate without an explicit mechanism. This is the central challenge NLP must solve.

A taxonomy by input–output structure

Category	Input	Output
Text classification	text	discrete label(s)
Sequence labelling	text	one label per token
Sequence-to-sequence	text	a different text
Question answering	question + context	answer (span / text)
Open-ended generation	prompt	arbitrary text

Spectrum of output complexity: from one integer (classification) to free-form sequences (generation). Each level adds expressiveness and evaluation difficulty.

Classification & sequence labelling

Text classification

One label per document.

- Sentiment analysis
- Spam detection
- Topic categorization
- Intent detection (chatbot)
- Toxicity / hate speech

Sequence labelling

One label per token.

- POS tagging
- Named Entity Recognition (NER)
- Slot filling

B-I-O scheme:
Apple/B-ORG was/O founded/O by/O Steve/B-PER Jobs/I-PER

Generation tasks

Sequence-to-sequence

- Machine translation
- Abstractive summarization
- Paraphrasing
- Grammar correction

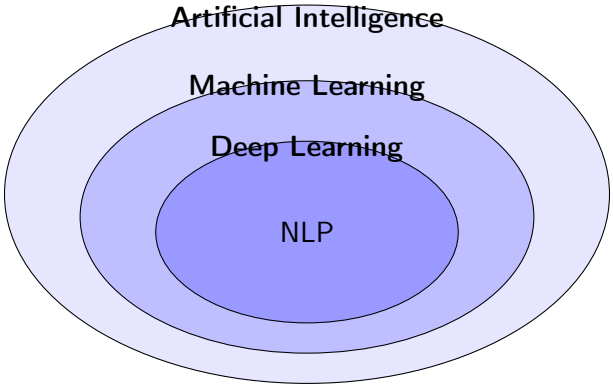
Question answering

- **Extractive**: span in context
- **Closed-book**: no context
- **Open-domain**: retrieve + read (RAG)
- **Generative**

Open-ended generation

The regime of large language models (LLMs): given a prompt, produce a fluent continuation. Evaluation is intrinsically harder because many outputs are acceptable.

The AI / ML / DL hierarchy



Strict inclusion: $\mathrm{DL} \subset \mathrm{ML} \subset \mathrm{AI}$. NLP is a domain that draws on all three across its history.

The four ingredients of any learned system

Learning quadruple $(D, \mathcal{F}, \mathcal{L}, \mathcal{O})$

- **Data** $D = \{(x_i, y_i)\}_{i=1}^{N}$ — training examples.
- **Model** $\mathcal{F} = \{f_\theta : \theta \in \Theta\}$ — parameterised functions.
- **Loss** $\mathcal{L}(\hat{y}, y)$ — penalty for bad predictions.
- **Optimizer** $\mathcal{O}$ — updates θ to reduce average loss.

Empirical Risk Minimization:

$$\theta^* \in \arg\min_\theta \frac{1}{N} \sum_{i=1}^{N} \mathcal{L}\big(f_\theta(x_i), y_i\big).$$

Three learning paradigms

Paradigm	**Labels?**	**Goal**	**NLP example**
Supervised	Provided	Predict $y \mid x$	Spam detection
Unsupervised	None	Find structure	Topic clustering
Self-supervised	Manufactured	Learn representations	Next-token prediction

Why self-supervision dominates modern NLP

Self-supervised pretraining converts *any* large unlabelled text into a training signal. The model learns general linguistic competence and transfers to downstream tasks via fine-tuning or prompting.

Discriminative vs. generative

Discriminative

Models $P(y \mid x)$.

- Decision boundary
- LR, SVM, BERT, CRF
- Predicts labels directly
- Cannot generate text

Generative

Models $P(x)$ or $P(x, y)$.

- Distribution over inputs
- N-grams, GPT, T5, VAEs
- Can generate samples
- Can score likelihoods

Bayes bridge (one-way)

Generative $\Rightarrow$ Discriminative via $P(y \mid x) = \frac{P(x|y)P(y)}{\sum_{y'} P(x|y')P(y')}$. The reverse direction is impossible.

The modern blur

LLMs are generative models doing discriminative tasks

A decoder-only Transformer (GPT-4, Claude, LLaMA) is trained generatively (next-token prediction) but can be *prompted* to solve any classification task:

> *"Classify the following review as positive or negative: . . . "*

The model produces the answer as a continuation, exploiting its general linguistic competence.

$\Rightarrow$ A single generative model handles the full task taxonomy.

Four eras of NLP

Recurring pattern

Each era **addresses the previous one's failure mode**: rules don't scale → statistics; counts lack semantics → embeddings; embeddings are static → contextualisation.

Era 1 — Rule-based (1950–1980)

- Linguists hand-write rules; expert systems encode facts.
- Formal grammars, regex, dictionary-based morphological analysers.
- Examples: ELIZA (1966), SHRDLU (1972), Georgetown–IBM MT (1954).

Limitations

Brittle, language-specific, expensive maintenance, cannot handle ambiguity probabilistically.

Era 2 — Statistical (1990–2010)

N-gram language model

$$P(w_1, \ldots, w_T) \approx \prod_{t=1}^{T} P(w_t \mid w_{t-n+1}, \ldots, w_{t-1}).$$

Probabilities estimated by counting in a corpus.

- Hidden Markov Models for POS tagging.
- Bag-of-Words, TF–IDF for retrieval.
- Maximum-entropy and log-linear classifiers.

Limitations

Sparse data, fixed context window, no semantic similarity.

Era 3 — Word embeddings (2013–2017)

Idea

Each word $\rightarrow$ a dense vector in $\mathbb{R}^d$ (typically $d = 50$–300). Words with similar contexts get similar vectors.

- **Word2Vec** (Mikolov 2013): skip-gram, CBOW.
- **GloVe** (Pennington 2014): co-occurrence matrix factorization.
- **FastText** (Bojanowski 2017): sub-word n-grams, no OOV.

Vector arithmetic

$$\overrightarrow{\text{king}} - \overrightarrow{\text{man}} + \overrightarrow{\text{woman}} \approx \overrightarrow{\text{queen}}$$

Limitations

Static (one vector per word, regardless of context).

Era 4 — Transformers & LLMs (2017–)

- Vaswani et al. (2017): *"Attention Is All You Need"*.
- BERT (2018): masked LM, encoder-only, fine-tuned downstream.
- GPT-1, 2, 3, 4: causal LM, decoder-only, scaled to billions of parameters.

Hallmarks

Contextual embeddings • **self-supervised** pretraining at Web scale • **foundation** models adapted to many tasks • **scaling laws** • chain-of-thought reasoning • RLHF alignment.

Chapter 2

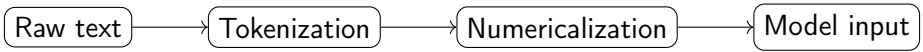
Text Representation and Evaluation

From raw text to vectors and metrics

Tokenization: the first critical step

Why tokenize?

Models cannot process raw text directly. Tokenization converts text into **discrete units** that can be represented numerically.



"I love NLP" → `[I, love, NLP]` → `[42, 318, 7953]`

Three granularities

Level	Vocab size	Sequence length	OOV
Word	Very large (10^5–10^6)	Short	`[UNK]`
Character	Tiny (~ 100)	Very long	None
Sub-word (BPE)	Moderate (10^4–10^5)	Moderate	Robust

Modern LLMs

GPT, BERT, LLaMA — all use **sub-word** tokenization (BPE, WordPiece, SentencePiece). Best trade-off between vocabulary size and sequence length.

Byte-Pair Encoding (BPE)

Algorithm:

1. Start with character-level vocabulary.
2. Count adjacent symbol pairs in the corpus.
3. Merge the most frequent pair into a new symbol.
4. Repeat until vocabulary reaches target size.

Example merge

Corpus: `newest, newest, widest, lowest`
Most frequent pair: `e + s` $\rightarrow$ merge into `es`.
Then: `es + t` $\rightarrow$ `est`, etc.

New word `lowest` tokenizes as `low + est` (sub-words seen during training).

Bag-of-Words and TF–IDF

Bag-of-Words

Document $\rightarrow$ vector of word **counts**.
"the cat sat" $\rightarrow [1, 1, 1, 0, \dots]$

TF–IDF

Down-weight common words, up-weight rare ones:

$$\mathrm{tfidf}(w, d) = \mathrm{tf}(w, d) \times \log \frac{N}{\mathrm{df}(w)}.$$

Properties

- Simple, fast, interpretable
- Strong baseline for classification & search
- Works well on small corpora

Limitations

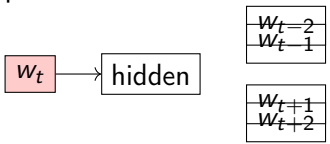
- No word order
- No semantic similarity (*cat* $\perp$ *kitten*)
- Very high dimension

Word2Vec — distributional hypothesis

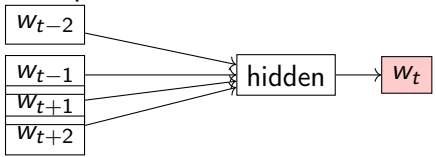
Distributional hypothesis (Firth, 1957)

"You shall know a word by the company it keeps."
Words appearing in similar contexts have similar meanings.

Skip-gram
Centre → predict context



CBOW
Context → predict centre



From rules to embeddings: a synthetic comparison

Approach	Representation	Semantics	Scalability	Era
Rules	Symbolic	Manual	Low	1950
N-grams / BoW	Counts	No	Medium	1990
TF–IDF	Weighted counts	Partial	Medium	1990
Word2Vec	Dense vectors	Yes	High	2013
GloVe	Dense vectors	Yes	High	2014
FastText	Sub-word vectors	Yes	High	2017
Transformers	Contextual vectors	Yes	Very high	2017+

Next: contextual embeddings

The static-vs-contextual distinction motivates the move to Transformers (Part 2 of this course).

Measuring similarity

Cosine (most common)

$$\cos(\mathrm{a}, \mathrm{b}) = \frac{\mathrm{a}^\top \mathrm{b}}{\|\mathrm{a}\| \, \|\mathrm{b}\|} \in [-1, 1]$$

Magnitude-invariant. Preferred for text.

Dot product

$$\mathrm{a}^\top \mathrm{b}$$

Fast; sensitive to magnitude.

Euclidean

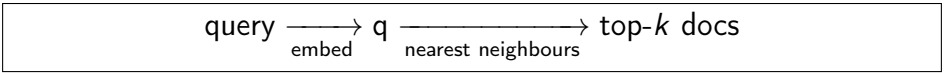
$$d_2(\mathrm{a}, \mathrm{b}) = \|\mathrm{a} - \mathrm{b}\|_2$$

Sensitive to magnitude AND direction.

Rule of thumb

Use cosine for text/TF–IDF/embeddings.
Use Euclidean for k-means after ℓ_2-normalisation.
Use dot product for speed (after normalisation, equivalent to cosine).

The retrieval pattern



Applications:

- Semantic search engines
- Document deduplication
- Recommendation systems
- **Retrieval-augmented generation (RAG)** for LLMs
- Cross-lingual matching

At scale: approximate nearest-neighbour libraries (FAISS, HNSW, ScaNN).

Why measure?

"What is not measured cannot be improved."

Classification (discriminative)

- Accuracy
- Precision, Recall
- F_1 score

Generation (generative)

- Perplexity (language modelling)
- BLEU (translation)
- ROUGE (summarisation)

Important

No single metric tells the whole story. Always report several and consider the task context.

Classification metrics

Confusion matrix

	Pred +	Pred −
True +	TP	FN
True −	FP	TN

Formulas

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{Total}}$$

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

F_1 score

Harmonic mean of P and R:

$$F_1 = 2 \cdot \frac{P \cdot R}{P + R}$$

High only when **both** are high.

Class imbalance!

With 95% negative examples, "always predict negative" achieves 95% accuracy — but is useless.

$\Rightarrow$ Always inspect P, R, F_1.

Precision vs Recall: which to optimize?

Library analogy

Precision: of the books I brought home, how many are about ML?
Recall: of all ML books in the library, how many did I bring home?

High Precision	High Recall	Balanced (F_1)
Spam detection	Medical screening	NER
Legal search	Fraud detection	Sentiment
		Most NLP tasks

Perplexity — evaluating language models

Definition

$$\mathrm{PPL} = \exp\left(-\frac{1}{N}\sum_{i=1}^{N}\log P(w_i \mid w_1, \ldots, w_{i-1})\right).$$

Intuition: If $\mathrm{PPL} = k$, the model is on average "hesitating between k equally likely words" at each position.

Lower is better. $\mathrm{PPL} = 1$ means perfect prediction.

Caveat

Low perplexity does **not** guarantee high-quality generation. A model can be confident yet repetitive.

BLEU — evaluating translation

Bilingual **E**valuation **U**nderstudy (Papineni et al., 2002).

Formula

$$\mathrm{BLEU} = \mathrm{BP} \cdot \exp\left(\frac{1}{N}\sum_{n=1}^{N} \log p_n\right),$$

where:

- p_n = modified n-gram precision (capped by reference counts).
- BP = brevity penalty (penalises too-short outputs).

Quick example

Reference: *The cat sat on the mat.*
Candidate: *The cat sat on the mat.* $\rightarrow$ BLEU = 1.0
Candidate: *The cat on the mat.* $\rightarrow$ BLEU drops (missing word).

ROUGE — evaluating summarisation

Recall-**O**riented **U**nderstudy for **G**isting **E**valuation (Lin, 2004).

ROUGE-N

$$\frac{|\, n\text{-grams}_c \cap n\text{-grams}_r \,|}{|\, n\text{-grams}_r \,|}$$

- ROUGE-1: unigrams
- ROUGE-2: bigrams

ROUGE-L

Based on the longest common subsequence (LCS).

Captures sentence-level structure without requiring contiguous matches.

BLEU vs ROUGE

BLEU = precision-oriented (translation).
ROUGE = recall-oriented (summarisation, where missing content is the failure mode).

Metrics summary

Metric	Task	Measures	Range
Accuracy	Classification	Overall correctness	$[0, 1]$
Precision	Classification	Prediction quality	$[0, 1]$
Recall	Classification	Coverage	$[0, 1]$
F_1	Classification	Harmonic mean P/R	$[0, 1]$
Perplexity	Language modelling	Predictive quality	$[1, \infty)$
BLEU	Translation	n-gram precision	$[0, 1]$
ROUGE-N	Summarisation	n-gram recall	$[0, 1]$
ROUGE-L	Summarisation	LCS recall	$[0, 1]$

Modern complementary metrics

BERTScore (semantic), **LLM-as-a-judge**, **RAGAS** (RAG eval), **MT-Bench** (instructions).

Take-aways

1. NLP is hard because of **ambiguity**, **context**, and **variability**.
2. Tasks span **classification, sequence labelling, seq-to-seq, QA, generation**.
3. NLP draws on **ML and DL**; modern systems are **self-supervised** and generative.
4. Each historical era **addressed the previous one's failure mode**.
5. Tokenization, vector representations and metrics are the **core toolkit** every practitioner must master.

What's next?

Lab notebooks (4 sessions)

- `lab01-tokenization.ipynb`
- `lab02-bow-tfidf-classifier.ipynb`
- `lab03-word2vec-embeddings.ipynb`
- `lab04-evaluation-metrics.ipynb`

Future parts

Part 2: From RNNs to Transformers (sequence models, attention, BERT, GPT)
Part 3: Pretraining and fine-tuning (transfer learning, instruction tuning, RLHF)
Part 4: Retrieval-Augmented Generation (RAG) and AI agents

Thank you!

Questions?

https://github.com/Aymenbenbrik/FundamentalsNLP